Central Device Controller:
The Device class serves as the central orchestrator. It initializes and connects all the major modules such as the PumpController, UserInterface, CGMReader, BatteryManager, InsulinReserve, DataLogger, and Profile management, ensuring all components work together.
PumpController:
Handles all insulin delivery operations. It's responsible for both bolus and basal rates. The class divides the insulin delivery process into time-based "ticks" so that insulin (bolus) is delivered gradually based on a specified rate. This was guided by our requirements to simulate a real-life pump that distributes the insulin dose over time.
BolusCalculator:
Determines the insulin dose based on inputs such as current glucose levels and carbohydrate intake. The BolusCalculator figures out how much insulin is needed and the PumpControllder manages how that dose is delivered.
UserInterface:
The UserInterface provides navigation between different screens (login, home, bolus calculator, settings, and history). It uses Qt's signal-slot mechanism for responsiveness and decoupled interactions.
Profile Management:
Manages by profile class (with CRUD operations), ensuring that personal delivery settings are enclosed and safely stored. This supports the use case around personalized insulin delivery. (see Use Cases 3004).

State Modeling with UML
The system's dynamic behaviors were modeled with UML state diagrams (referenced in Insulin Pump Simulator - UML State Diagrams.pdf). These diagrams helped us identify key states and transitions such as:

- Normal Operation
- Suspension/Resumption
- Emergency stop

This state-driven approach makes it easier to enforce safety protocols and ensures that the simulator behaves predictably under various conditions.

Safety and ErrorHandling
Fail-safe validation:

- In the PumpController, we added checks (eg., not pumping if in an emergency state or if bolus delivery is suspended) to avoid dangerous insulin over-delivery.

Alerts and Logging:

- The DataLogger captures every event (whether an error, a system alert, or routine activity) to support troubleshooting and historical review. This design supports requirements listed in our traceability matrix (see Traceability-Matrix-draft.pdf) where every safety-critical action is logged.

Use of Qt's Signal and Slot

- Error handling and UI updates are managed through signals and slots, which decouples the presentation layer (UserInterface) from back-end processes (eg., BatteryManager, CGMReader). This helps with safety and debugging.

Traceability and Requirement

The design decisions were not made in isolation but rather mapped directly to our project requirements and use cases. For example:

- UC3: Deliver Manual Bolus
  The BolusCalculator computes the dose, and the PumpController's deliverBolus() and pump() methods handle the gradual delivery over time.
- UC5: Monitor and Adjust Insulin Delivery
  The ControlIQAlgorithm class actively monitors CGM data and triggers adjustments via PumpController.
- UC7: Error handling & Alerts
  Components like BatteryManager and CGMReader include methods (such as alertLowBattery() and alertCGMDisconnected()) that integrate with the UserInterface for proactive error notifications.

Implementation Decisions

- Time Dependent Pumping: The PumpController's pump() method simulates insulin delivery in "ticks". Instead of delivering the entire bolus immediately, it calculates the amount per tick based on a rate.
- Direct Use of Shared Objects: Insulin Reserve, DataLogger and CGMReader are passed as pointers to the PumpController and other classes, ensuring a single source of truth.

Overall, our design emphasizes a modular, traceable, and safe system:

- The Device class remains the central controller, overseeing interactions among all components.
- PumpController is refined to deliver insulin over time in a realistic simulation.
- All design decisions are driven by clearly defined requirements and supported by UML diagrams and traceability matrices.